

오늘의 작업기록 — 2026년 7월 9일

재현 훈련 1호 — HealthController 빈 화면 제작성 (+ tmux 도입)

(복기 빛의 0을 깬 날 — 10분짜리 세션도 세션이다)

한눈에

항목	내용
성격	초단축 세션(~10분) — 재현 훈련 1호 + 환경 개선 1건
결과	① HealthController 기억 제작성 통과(교정 2곳) ② tmux 도입(세션 dev) ③ 개념 질문 3개 발제 — 소화는 미완(재독 예약)
교정 2곳	<code>package ...</code> 세미콜론 누락 · <code>@RestController {</code> — 애노테이션에 중괄호를 붙임
판단	노트북 반입 트랙 폐기(부대 내 핫스팟 불가 → 인터넷 없는 개발환경은 ROI 없음) → 질의 대상은 블루투스 키보드 하나로 축소
미완	애노테이션 3대 질문의 답변 소화 — 아래 §3에 답 요지 보존, 다음 세션 첫 5분 재독
다음	재진입점 유지 = 예외→HTTP 번역 · 문답집은 인쇄해 틈새 자료로

어제 "도구만 만들고 안 썼다"를 짚었고, 오늘 그 0을 깬다. 10분이 전부였지만 재현 1번을 실제로 돌렸고, 재현이 아니면 안 나왔을 구멍 두 개(세미콜론, 애노테이션에 중괄호)를 잡았다. 둘 다 "개념을 모름"이 아니라 "문법 디테일이 헛거움" — 정확히 재현 훈련이 잡으라고 있는 유형이다.

1. 재현 훈련 1호 — 실행과 판정

tmux 세션 안에서, 아무것도 안 보고:

```
nano ~/test-recall.java      # HealthController를 기억만으로 제작성
```

1차 결과 — 뼈대 전부 나옴, 교정 2곳:

```
package com.ledger.api      // x 세미콜론 누락 - 선언문은 ; 필수
...
@RestController {           // x 애노테이션에 중괄호 - 스티커는 블록이 아니다
    public class HealthController {
```

- 나온 것: import 2줄(GetMapping·RestController), `@GetMapping("/health")`, 메서드 시그니처, `return "ledger-api alive";` — 3일 전 처음 본 코드의 골격이 기억에서 전부 나왔다.
- 교정 ①: `package com.ledger.api;` — 문장(선언문)이므로 세미콜론으로 끝나야 한다.
- 교정 ②: 애노테이션은 단독 줄로 끝난다. `@RestController` 다음 줄에 `public class ... {`. 중괄호를 붙이면 여는 괄호가 하나 남아 컴파일 자체가 안 된다.

2차 제작성 — 통과. 괄호 짝(class 열고 → 메서드 열고 닫고 → class 닫고)까지 정상. 컴파일 가능한 형태 확인.

판정: 재현 통과. 복기 빛에 대한 걱정은 절반은 과장이었다 — 골격은 붙어 있고, 헛거움 건 문법 마감 디테일이다.

2. tmux — 오늘부터의 작업 방식

한 줄 정의: SSH가 끊겨도 죽지 않는 터미널 세션.

```
tmux new -s dev      # 세션 생성 (최초 1회)
tmux attach -t dev    # 끊긴 뒤 재접속 - 하던 화면 그대로 복귀
# Ctrl+b 누른 뒤 % → 좌우 분할 (한쪽 서버 로그, 한쪽 curl)
```

- 폰 작업의 두 고통(연결 끊기면 전멸 · 창 하나)을 동시에 해결. `> server.log 2>&1 &` 곡예의 상당 부분을 대체한다.
- 사지방에서도 `tmux attach` — 폰에서 하던 화면을 큰 모니터로 이어받는다. 두 기기가 한 작업대가 된다.

환경 판단 확정: 노트북 반입 트랙 폐기 — 부대 내 핫스팟 불가이므로 반입해도 인터넷 없는 개발환경 = 비용 대비 무의미. 남은 질의는 블루투스 키보드 반입 가능 여부 하나. IntelliJ 성능 튜닝 과제도 폐기(초기화되는 공용 저사양 PC에 투자 안 함) — 사지방 PC는 SSH 단말/열람용으로 역할 재정의.

3. 애노테이션 3대 질문 — 발제와 답 요지 (★소화 미완, 재독용)

재현 중 나온 진짜 질문 셋. 답의 관통선 하나: **달라진 건 자바가 아니라 호출자다**. Main 시절엔 내가 직접 불렀으니 아무한테 알릴 게 없었고, 지금은 스프링이 대신 부르므로 "누가 HTTP 담당인지, 어느 URL이 어느 메서드인지"를 스프링에게 알려야 한다 — 그 언어가 @ 다.

이 절의 기준 코드 — HealthController 전문 (7/6 작성, 질문 3개가 전부 이 파일 애기다):

```
package com.ledger.api;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HealthController {

    @GetMapping("/health")
    public String health() {
        return "ledger-api alive";
    }
}
```

Q1. import로 똑같이 땡겨왔는데 왜 애들만 @를 붙이나? → import가 가져온 이름의 **종족**이 다르다. **Map** 은 클래스(재료)라 **new** 해서 쓰고, **RestController** 는 애노테이션(스티커)이라 붙이는 것만 된다. @가 "붙인다"는 문법이다. (mini-ledger에 @가 없던 이유 = 스티커를 읽을 주체(스프링)가 없었다.)

Q2. 왜 @RestController와 @GetMapping이 "두 층"인가? → @GetMapping이 @RestController "안에" 있는 게 아니라, **클래스 안에 메서드가 있는 것**(Bank 안에 transfer 있던 그 구조)이고 스티커가 각자 자기 층에 붙은 것. 역할이 다르다: 클래스 스티커 = 스캔이 찾는 **등록 단위**("이 건물은 HTTP 부서"), 메서드 스티커 = 그 안의 **개별 창구**("이 창구는 /health 업무"). 한 클래스가 창구 여러 개를 가질 수 있으므로 URL은 메서드 단위가 맞다.

[7/10 보강 — 추가 질문 2개의 답]

"스프링이 @를 어떻게 읽어내나?" — 자바는 클래스에 붙은 애노테이션을 **기계가 조회할 수 있는 명찰**로 저장한다(컴파일된 .class 파일 안에 명찰이 같이 박힘). 부팅 때 스프링 스캐너가 전 클래스를 돌며 말 그대로 "너 @RestController 붙어있나?"를 조회하고, 붙어있으면 등록 → 그 클래스의 메서드들을 훑으며 "@GetMapping 붙어있나? 값이 뭐냐?"를 조회해 **"/health"** 를 읽고 URL 표에 **GET /health → health()** 한 줄을 기록한다. @는 장식이 아니라 **조회 가능한 명찰**이고, 부팅 = 전 클래스 명찰 검사다.

"Map이랑 RestController, 둘 다 import인데 종족이 어떻게 다르냐?" — 종족은 **태어날 때(정의될 때)** 정해진다. 각자의 소스(출생신고)를 까보면:

```
public interface Map<K,V> { ... } // Map의 출생신고: 재료(인터페이스/클래스 계열)
public @interface RestController { ... } // RestController의 출생신고: @interface = 애노테이션
```

@interface 라는 **별도 키워드**로 정의된 것만 애노테이션이다. import는 "이 이름을 이 파일에서 쓰겠다"고 주소록에 올리는 행위일 뿐 — 이름이 가리키는 실체의 종족은 출생 때 이미 정해져 있고 import가 바꾸지 못한다. 그래서 **new RestController()** (재료처럼 쓰기)도, **@Map** (스티커처럼 쓰기)도 컴파일 에러다. **사용법은 종족을 따른다.**

Q3. ("/health") 는 왜 넣냐? → 애노테이션은 값을 받을 수 있다 — **스티커에 쓰는 글자**. @GetMapping만으로는 "GET을 받는다"까지고, **"/health"** 가 **주소판**이다. 부팅 때 URL 표에 **GET /health → health()** 로 등록되고, 요청이 오면 표에서 찾아 호출한다. (7/7의 **@PostMapping("/transfer")** 가 같은 패턴.)

★ **미완 표시:** 시간 부족으로 이 답들을 소화 못 하고 세션 종료. **다음 세션 첫 5분 = 이 § 3 재독부터.** (문답집 Q10~Q13이 같은 내용의 확장판이다.)

4. 로드맵 위치와 다음

- 재진입점 유지: **예외→HTTP 번역** (a=60, curl 한 방이면 500 관찰 — 진입 비용 최저 상태 유지 중)
- 복기 잔여: § 3 재독(5분) → 문답집 부록 체크 2번(record 손코드)부터 이어서 · 7/5분 재현(원자성·checkBalance) 예약 유지
- m106(CodeCrafters Redis) 7월 후반 · m107 월말

5. 검증 질문 (다음 복기용 — 답 적지 말 것)

- 재현 1호에서 틀린 두 곳은? 각각 왜 틀렸고, 애노테이션 줄의 올바른 형태는?
- "달라진 건 자바가 아니라 호출자다" — 이 문장으로 @의 존재 이유를 설명하라.
- import한 Map과 import한 RestController의 사용법이 다른 이유는? (종족)
- 클래스 스티커와 메서드 스티커의 역할 차이는? 왜 URL은 메서드 단위인가?
- tmux의 두 효용과, 재접속 명령은?

6. 회고

10분짜리 세션이었지만 어제의 처방("쉬든가, 10분 치든가 — 메타작업만 무효")을 그대로 이행한 날이다. 도구를 하나 더 만드는 대신 재현을 실제로 돌렸고, 재현이 아니면 영영 안 보였을 구멍 — 세미콜론, 그리고 애노테이션을 블록으로 착각한 것 — 을 잡았다. 특히 두 번째가 값지다: @RestController에 종괄호를 붙였다는 건 애노테이션을 "코드 구조"로 오해하고 있었다는 뜻이고, 그 오해가 곧바로 오늘의 질문 셋("왜 @냐, 왜 두 층이냐, 괄호 안은 뭐냐")으로 이어졌다. 재현 훈련이 약점을 드러내고, 약점이 질문을 만들고, 질문이 다음 공부의 좌표가 되는 — 사이클이 한 바퀴 돈 셈이다.

솔직한 미완도 남긴다: 그 질문들의 답을 시간에 쫓겨 소화하지 못했다. 그래서 § 3에 답 요지를 보존해뒀고, 다음 세션의 첫 5분을 거기에 배정한다. 이해를 못 한 채 끝난 게 아니라, 이해할 지점을 정확히 짚어두고 끝난 것 — 짧은 날은 그거면 된다.

한 줄: 10분으로 복기 빛의 0을 깼다 — HealthController 재현 통과(교정 2곳), tmux 도입, 그리고 "왜 @인가"라는 좋은 질문 셋을 좌표로 남김.